

**FEATURE DRIFT DETECTION TECHNIQUES VIA
ADVERSARIAL VALIDATION**

NUSRAT BEGUM

**A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF SCIENCE (COMPUTER SCIENCE)
FACULTY OF GRADUATE STUDIES
MAHIDOL UNIVERSITY
2026**

COPYRIGHT OF MAHIDOL UNIVERSITY

Thesis
entitled
**FEATURE DRIFT DETECTION TECHNIQUES VIA
ADVERSARIAL VALIDATION**

.....
Ms. Nusrat Begum
Candidate

.....
Asst. Prof. Thanapon Noraset,
Ph.D. (Computer Science)
Major advisor

.....
Assoc. Prof. Suppawong Tuarob,
Ph.D. (Computer Science)
Co-advisor

.....
Prof. Dr. Chartchalerm
Isarankura-Na-Ayudhya,
Ph.D. (Medical Technology)
Dean
Faculty of Graduate Studies
Mahidol University

.....
Assoc. Prof. Boonsit Yimwadsana,
Ph.D. (Electrical Engineering)
Program Director
Master of Science Programme
in Computer Science
Faculty of Information and
Communication Technology
Mahidol University

Thesis
entitled
**FEATURE DRIFT DETECTION TECHNIQUES VIA
ADVERSARIAL VALIDATION**

was submitted to the Faculty of Graduate Studies, Mahidol University
for the degree of Master of Science (Computer Science)

on
January 5, 2021

.....
Ms. Nusrat Begum
Candidate

.....
Asst. Prof. Thanawin Rakthanmanon,
Ph.D. (Computer Science)
Chair

.....
Asst. Prof. Thanapon Noraset,
Ph.D. (Computer Science)
Member

.....
Assoc. Prof. Suppawong Tuarob,
Ph.D. (Computer Science)
Member

.....
Prof. Dr. Chartchalerm
Isarankura-Na-Ayudhya,
Ph.D. (Medical Technology)
Dean
Faculty of Graduate Studies
Mahidol University

.....
Dr. Pattanasak Mongkolwat,
Ph.D. (Computer Science)
Dean
Faculty of Information and
Communication Technology
Mahidol University

ACKNOWLEDGEMENTS

Nusrat Begum

FEATURE DRIFT DETECTION TECHNIQUES VIA ADVERSARIAL VALIDATION

NUSRAT BEGUM 6638009 ITCS/M

M.Sc. (COMPUTER SCIENCE)

THESIS ADVISORY COMMITTEE: THANAPON NORASET, Ph.D., SUPPAWONG
TUAROB, Ph.D.

ABSTRACT

IMPLICATION OF THE THESIS

**KEY WORDS : FEATURE DRIFT DETECTION / ADVERSARIAL VALIDATION /
MACHINE LEARNING / CONCEPT DRIFT**

34 pages

CONTENTS

	Page
ACKNOWLEDGEMENTS	iii
ABSTRACT	iv
LIST OF TABLES	viii
LIST OF FIGURES	ix
CHAPTER I INTRODUCTION	1
1.1 Motivation	1
1.2 Objectives	2
1.3 Expected Contributions	3
CHAPTER II LITERATURE REVIEW	5
2.1 Overview	5
2.2 Types of Drift	6
2.2.1 Drift by Nature of Change	6
2.2.2 Drift by Temporal Pattern	7
2.3 Classification of Drift Detection Methods	8
2.3.1 Supervised Drift Detection Methods	8
2.3.2 Unsupervised Drift Detection Methods	9
2.3.3 Hybrid and Ensemble Methods	12
2.4 Limitations of Existing Methods	12
2.5 Related Work: Adversarial Validation	14
2.5.1 Concept and Origins	14
2.5.2 Industrial Applications	14
2.6 Gap in the Literature	15
CHAPTER III METHODOLOGY	17
3.1 Problem Formulation	17
3.2 Core Components of the Framework	18
3.2.1 Streaming Window Management	18

CONTENTS (cont.)

	Page
3.2.2 Adversarial Validation as Discriminative Drift Detector	19
3.2.3 Detection Statistic and Decision Rules	20
3.2.4 Model Adaptation Strategies	21
3.3 Datasets	21
3.4 Baseline Drift Detectors	22
3.5 Evaluation Metrics	23
3.6 Experimental Design	24
3.7 Practical Considerations	25
CHAPTER IV RESULTS	26
CHAPTER V DISCUSSION	27
CHAPTER VI CONCLUSION	28
BIOGRAPHY	34

LIST OF TABLES

Table

Page

LIST OF FIGURES

Figure	Page
2.1 Comparison of no drift versus drift scenarios. Left: When distributions overlap, the adversarial classifier cannot distinguish data sources ($AUC \approx 0.5$). Right: When drift occurs, distributions shift and the classifier can distinguish sources ($AUC \gg 0.5$).	5
3.1 Overview of the methodology for feature drift detection via adversarial validation.	17

CHAPTER I

INTRODUCTION

1.1 Motivation

Machine learning models deployed in real-world environments frequently encounter evolving data distributions over time. According to Lu et al. (2018) and Gama et al. (2014), these changes—collectively referred to as distribution shift or drift—can manifest as changes in input features (covariate drift) or in the relationship between inputs and labels (concept drift), causing model performance to deteriorate if left unmanaged. Lu et al. (2018) and Widmer and Kubat (1996) emphasize that detecting such changes early is essential for maintaining reliable model performance, reducing operational risk, and avoiding accumulating technical debt in production systems (Bayram et al., 2022; Sculley et al., 2015).

Conventional model monitoring strategies react to drift only after observable performance degradation, which by design means that the system has already suffered from poor predictions on new data (Widmer and Kubat, 1996). In production contexts where ground-truth labels may be delayed, unavailable, or costly to obtain, this post-hoc reactionary approach is often inadequate (Lu et al., 2018; Souza et al., 2020). According to Lu et al. (2018) and Bayram et al. (2022), early drift detection that anticipates distribution changes before significant performance loss is therefore a practical priority in dynamic machine learning systems.

Current drift detection methods are varied. Traditional statistical methods employ hypothesis tests—such as the Kolmogorov–Smirnov test, chi-squared test, or Kullback–Leibler divergence—to detect distribution changes by comparing incoming and reference data (Rabanser et al., 2019; Raab et al., 2020). Bifet and Gavaldà (2007) developed adaptive windowing methods like ADWIN that dynamically adjust how much historical data is retained for comparison. Supervised error-based detectors like DDM and EDDM monitor model performance metrics over time, but require access to ground-truth

labels (Gama et al., 2004; Baena-García et al., 2006). Prediction-based detectors track model outputs for changes in confidence, entropy, or prediction distributions without necessarily requiring labels (Sethi and Kantardzic, 2017). Despite this diversity, several challenges remain: statistical tests can be sensitive to noise and may struggle in high-dimensional settings; supervised methods depend on label availability; prediction-based methods may be confounded by model calibration issues; and hybrid approaches often require engineered context variables that may not generalize across domains (Lu et al., 2018; Webb et al., 2016).

To address these gaps, we propose recasting feature drift detection as a discriminative learning problem: treating the comparison between historical and newly arriving data as a binary classification task. If an auxiliary classifier can reliably separate data from the two time periods based solely on features, it indicates a meaningful distributional change. This approach, known as adversarial validation, has been used in data science to detect and quantify dataset shift Pan et al. (2020); Qian and Rao (2021) but has not been systematically developed into a general feature drift detector for streaming or window-based data monitoring.

According to Lopez-Paz and Oquab (2017), this approach offers several key advantages for drift detection: it is label-independent (does not require ground-truth labels for new data), model-agnostic (can be applied regardless of the primary prediction model), makes no assumptions about which features or contexts are important, and can detect complex high-dimensional, non-linear distributional shifts that may evade univariate or marginal statistical tests.

1.2 Objectives

The primary objective of this thesis is to investigate adversarial validation as a feature (covariate) drift detection technique for machine learning models deployed in dynamic environments. Specifically, this work aims to:

1. Formalize feature drift detection as a discriminative two-sample learning problem by reframing the comparison between historical and incoming

data distributions as a binary classification task using adversarial validation.

2. Develop streaming and sliding window procedures for applying adversarial validation in practical drift detection settings without relying on immediate access to ground-truth labels or predefined context variables.
3. Systematically benchmark adversarial validation against established drift detection methods—including statistical tests such as ADWIN Bifet and Gavaldà (2007) and KSWIN Raab et al. (2020), error-based supervised detectors such as DDM Gama et al. (2004) and EDDM Baena-García et al. (2006), and distribution-based methods—across synthetic and real-world datasets exhibiting different types of drift.
4. Quantify detection performance characteristics including detection delay (lead time), false positive rate, true positive rate, sensitivity, and stability under various drift patterns (abrupt, gradual, incremental, recurring).
5. Evaluate practical applicability and scalability in realistic MLOps settings, including hyperparameter sensitivity (window sizes, classifier choices, thresholds), computational efficiency, and integration with model retraining strategies.

1.3 Expected Contributions

This thesis seeks to contribute:

1. A formal adversarial validation framework for unsupervised feature drift detection that directly assesses distributional separability between reference and current data, enabling label-free and model-agnostic monitoring.
2. A systematic benchmark evaluation of adversarial validation as a drift detector, situating it within the established taxonomy of drift detection methods and assessing its performance across controlled synthetic and realistic environments—an aspect not explored in prior work.

3. Empirical comparison with established drift detection methods, including statistical tests, error-based approaches, and distribution-based methods, highlighting differences in sensitivity, robustness, detection behavior, and computational requirements.
4. Analysis of lead time and stability of adversarial validation–based drift signals, demonstrating its potential to provide earlier and more reliable warnings of distributional change than methods that rely on delayed labels or calibrated prediction uncertainty.
5. Practical guidance for deployment in MLOps pipelines, including recommendations for windowing strategies, classifier selection, threshold determination, and interpretability of drift signals in high-dimensional feature spaces.

CHAPTER II

LITERATURE REVIEW

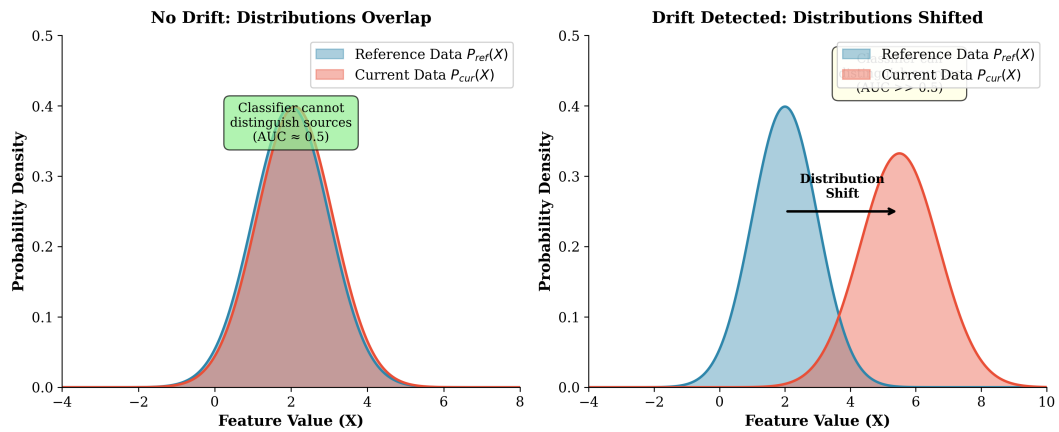


Figure 2.1: Comparison of no drift versus drift scenarios. Left: When distributions overlap, the adversarial classifier cannot distinguish data sources ($AUC \approx 0.5$). Right: When drift occurs, distributions shift and the classifier can distinguish sources ($AUC > 0.5$).

2.1 Overview

Drift detection addresses the challenge that arises when the statistical characteristics of data evolve over time in a manner that impacts the validity of a machine learning model (Lu et al., 2018; Gama et al., 2014). According to Bayram et al. (2022), when users, environments, data sources, or the relationships between labels and features change, model accuracy can diminish. Gama et al. (2014) and Sculley et al. (2015) emphasize that drift detection identifies this modification early, allowing practitioners to adapt, retrain, or update the model accordingly. A robust understanding of drift types and detection methods is essential for designing effective monitoring systems.

2.2 Types of Drift

According to Lu et al. (2018) and Gama et al. (2014), drift can be categorized along two main dimensions: by the nature of the change (what component of the data-generating process is affected) and by the temporal pattern of the change (how the drift manifests over time).

2.2.1 Drift by Nature of Change

Based on which component of the joint distribution $P(X, y)$ changes, Lu et al. (2018), Gama et al. (2014), and Tsymbal (2004) classify drift into the following types:

Concept Drift (Real Drift): According to Lu et al. (2018) and Žliobaitė (2010), the conditional distribution $P(y|X)$ changes over time while the feature distribution $P(X)$ may or may not change. This represents a change in the underlying relationship between inputs and outputs—the “concept” the model has learned becomes invalid. For example, Gama et al. (2014) note that the factors that determine creditworthiness may evolve due to economic shifts, even if the applicant characteristics (features) remain similar. Lu et al. (2018) emphasize that concept drift directly degrades model performance because the learned decision boundaries no longer reflect the true input-output relationship.

Covariate Drift (Feature Drift / Virtual Drift): According to Tsymbal (2004) and Shimodaira (2000), the marginal distribution of input features $P(X)$ changes over time while the conditional relationship $P(y|X)$ remains stable. The ground-truth concept does not change, but the model may encounter inputs outside its training distribution. Tsymbal (2004) provides an example: a fraud detection model trained on transaction patterns from one demographic may receive data from a different user population, even though the definition of fraud remains unchanged. Shimodaira (2000) explains that covariate drift can degrade model performance because the model is exposed to feature regions where it may not generalize well.

Prior Probability Drift (Label Shift): According to Tsymbal (2004), the marginal distribution of labels $P(y)$ changes over time while $P(X|y)$ remains stable. This occurs when the prevalence of different classes changes—for instance, a seasonal

increase in a particular disease affecting the class balance in medical diagnosis. Lu et al. (2018) note this is sometimes called class imbalance shift.

Note on Terminology: Lu et al. (2018) observe that the literature uses varied terminology. According to Gama et al. (2014), “concept drift” is sometimes used as an umbrella term for any distribution change affecting model performance, while Tsymbal (2004) reserves it specifically for changes in $P(y|X)$. This thesis focuses primarily on covariate (feature) drift detection, as it enables early warning without requiring labels.

2.2.2 Drift by Temporal Pattern

Based on how drift manifests over time, Lu et al. (2018), Gama et al. (2014), and Žliobaitė (2010) categorize drift as follows:

- **Abrupt (Sudden) Drift:** According to Gama et al. (2014), this is a rapid, discrete change where the data distribution shifts quickly from one state to another within a short time period.
- **Gradual Drift:** Lu et al. (2018) describe this as a new distribution progressively replacing the old one over an extended duration, with samples from both distributions interleaved during the transition period.
- **Incremental Drift:** According to Žliobaitė (2010), small but continuous changes accumulate over time, eventually resulting in a significant distributional shift.
- **Recurring (Seasonal) Drift:** Lu et al. (2018) and Gama et al. (2014) describe this as previously observed distributions reappearing periodically due to cyclical patterns, such as seasonal effects in retail or recurring user behavior patterns.

According to Gama et al. (2014), understanding drift patterns is important for detector design: abrupt drift requires rapid response, gradual drift requires sustained sensitivity, and recurring drift benefits from mechanisms that can recognize previously seen distributions.

2.3 Classification of Drift Detection Methods

Based on comprehensive surveys of drift detection literature (Lu et al., 2018; Gama et al., 2014; Tsymbal, 2004; Hinder et al., 2024), drift detection methods can be classified by multiple dimensions. According to Lu et al. (2018), the most fundamental distinction is based on label availability: whether the method requires access to ground-truth labels (supervised) or operates without them (unsupervised).

2.3.1 Supervised Drift Detection Methods

According to Gama et al. (2014) and Gama et al. (2004), supervised drift detection methods require access to ground-truth labels and primarily monitor model performance or prediction errors to detect distributional changes. Gama et al. (2004) and Baena-García et al. (2006) explain that these methods operate under the assumption that unexpected changes in error characteristics—such as increases in error rate, loss magnitude, variance, or cumulative deviation—are indicative of underlying drift.

Key supervised drift detection algorithms include:

Drift Detection Method (DDM): According to Gama et al. (2004), DDM monitors the online error rate and its standard deviation. When these statistics exceed predefined thresholds derived from binomial distribution bounds, DDM signals a warning or drift state. Gama et al. (2004) note that DDM is effective for abrupt drift but may be slow to detect gradual changes.

Early Drift Detection Method (EDDM): Baena-García et al. (2006) extend DDM by tracking the distance between consecutive classification errors rather than just the error rate. According to Baena-García et al. (2006), this modification improves sensitivity to gradual drift by detecting when errors cluster more closely together.

Page–Hinkley Test: Page (1954) developed this sequential analysis technique using cumulative sums to detect changes in the mean of a monitored signal. When cumulative deviation exceeds a threshold, drift is indicated.

Hoeffding Drift Detection Methods (HDDM-A, HDDM-W): According to Frias-Blanco et al. (2015), these methods use Hoeffding’s inequality to provide statistically grounded drift decisions with theoretical guarantees. HDDM-A detects abrupt

changes using moving averages, while HDDM-W is designed for gradual drift using window-based statistics.

Limitations: According to Lu et al. (2018) and Sethi and Kantardzic (2017), supervised methods fundamentally require labeled data, which may be delayed, expensive, or unavailable in many production scenarios. Sethi and Kantardzic (2017) notes that they also react to drift after it has already caused performance degradation, rather than detecting distributional change proactively.

2.3.2 Unsupervised Drift Detection Methods

According to Tsymbal (2004) and Hinder et al. (2024), unsupervised drift detection methods do not require ground-truth labels and instead monitor changes in data distribution, model outputs, or internal representations. Sethi and Kantardzic (2017) emphasizes that these methods enable earlier detection of distributional shifts—before performance degradation becomes observable—making them particularly valuable in production environments with label delays.

2.3.2.1 Statistical Distribution-Based Methods

According to Rabanser et al. (2019) and Raab et al. (2020), statistical test-based methods detect drift by directly examining changes in data distributions through hypothesis testing or distance-based measures. Bifet and Gavaldà (2007) describe a common strategy that compares a reference data window against a current data window to test whether they originate from the same distribution.

Key statistical drift detection methods include:

Adaptive Windowing (ADWIN): According to Bifet and Gavaldà (2007), ADWIN dynamically maintains a sliding window and automatically shrinks it when statistically significant distributional change is detected. Bifet and Gavaldà (2007) emphasize that ADWIN provides theoretical guarantees on false positive and false negative rates and adapts its window size based on observed change.

KSWIN: Raab et al. (2020) developed this method that applies the Kolmogorov–Smirnov two-sample test over sliding windows to detect changes in one-dimensional feature distributions. According to Raab et al. (2020), KSWIN is non-parametric and makes minimal assumptions about the data distribution.

Chi-Squared Test-Based Detectors: According to Lu et al. (2018), these use the chi-squared test to compare categorical feature distributions or discretized continuous features between time periods.

Kullback–Leibler (KL) Divergence and Other Divergence Measures: Dasu et al. (2006) describe methods that quantify the “distance” between probability distributions. KL divergence, Jensen–Shannon divergence, and Hellinger distance can all be used to measure distributional shift.

Maximum Mean Discrepancy (MMD): According to Gretton et al. (2012), MMD is a kernel-based two-sample test that compares distributions by mapping them into a reproducing kernel Hilbert space. Gretton et al. (2012) note that MMD can capture complex multivariate distributional differences.

Strengths and Limitations: According to Rabanser et al. (2019), statistical tests are interpretable, theoretically grounded, and label-free. However, Lu et al. (2018) and Webb et al. (2016) note that they can be sensitive to noise (causing false alarms), may struggle in high-dimensional spaces due to the curse of dimensionality, and typically test marginal distributions rather than joint distributions.

2.3.2.2 Model-Based Discriminative Methods

According to Pan et al. (2020), Lopez-Paz and Oquab (2017), and Sethi and Kantardzic (2017), discriminative drift detection methods train an auxiliary model to distinguish between reference and current data distributions. Lopez-Paz and Oquab (2017) explain that the core idea is to frame drift detection as a classification problem: if a classifier can accurately predict whether each instance originates from the reference or current distribution, the distributions must be distinguishably different.

Key approaches include:

Domain Classifier / Adversarial Validation: According to Pan et al. (2020) and Qian and Rao (2021), this approach trains a binary classifier (e.g., logistic regression, random forest, gradient boosting) to discriminate samples from reference vs. current windows. Pan et al. (2020) note that high discriminative performance (measured by accuracy, AUC, or similar metrics) indicates significant distributional shift.

Discriminative Drift Detector (D3): Sethi and Kantardzic (2017) developed this unsupervised method that uses discriminative classifiers trained on data subsets to

detect concept drift without access to true labels.

Margin-Based and Density-Based Discriminators: According to Ditzler and Polikar (2011), these methods monitor changes in classifier margin densities or decision boundary proximity to detect distributional shifts.

Strengths: According to Lopez-Paz and Oquab (2017), discriminative methods can capture high-dimensional, non-linear distributional shifts that marginal univariate tests may miss. They require no labels for the drift detection task itself (only pseudo-labels indicating data source). Pan et al. (2020) note that they naturally aggregate information across all features.

Limitations: According to Sethi and Kantardzic (2017), they require training a classifier at each monitoring step, which has computational cost; performance depends on classifier choice and hyperparameters; and interpretation of drift signals may be less intuitive than statistical test p-values.

2.3.2.3 Representation-Based Methods

According to Rabanser et al. (2019) and Liu et al. (2020), representation-based methods detect drift by monitoring changes in learned representations or embeddings rather than raw feature distributions:

Embedding Distribution Shift: Rabanser et al. (2019) describe computing statistical distances (cosine similarity, Wasserstein distance, MMD) between reference and current embedding distributions derived from neural network intermediate layers.

Deep Learning Representation Monitoring: According to Long et al. (2016), this approach tracks stability of learned representations across time, leveraging insights from domain adaptation and transfer learning literature.

Liu et al. (2020) note that these methods are particularly relevant for deep learning systems where raw feature distributions may not reflect semantically meaningful changes.

2.3.2.4 Prediction-Based / Output-Based Methods

According to Sethi and Kantardzic (2017) and Gözüaık et al. (2019), prediction-based methods detect drift by monitoring changes in model outputs over time without requiring evaluation of prediction correctness:

Prediction Distribution Shift: Gözüaık et al. (2019) describe monitoring

changes in the distribution of model predictions (class probabilities, regression outputs) over time.

Uncertainty / Entropy Monitoring: According to Guo et al. (2017), this approach tracks prediction entropy or uncertainty scores under the assumption that rising uncertainty reflects exposure to out-of-distribution data.

Confidence Monitoring: Guo et al. (2017) also describe tracking changes in model confidence levels and softmax output distributions.

Limitations: According to Guo et al. (2017), these methods depend on model calibration. If the model is poorly calibrated—which is common for modern deep learning models—Ovadia et al. (2019) note that changes in confidence may not reliably indicate drift.

2.3.3 Hybrid and Ensemble Methods

According to Gama et al. (2014) and Krawczyk et al. (2017), hybrid approaches integrate multiple sources of information—both supervised and unsupervised signals—to improve drift detection robustness:

Context-Aware Drift Detection: Krawczyk et al. (2017) describe combining model predictions, user context, operational signals, and metadata to compute context-weighted drift scores.

Multi-Signal Ensemble Methods: According to Krawczyk et al. (2017), these integrate predictions, embeddings, operational KPIs, and metadata into a unified drift score.

Drift Detection Ensembles: Krawczyk et al. (2017) note that these combine multiple base detectors using voting, weighting, or meta-learning to reduce both false alarms and missed detections.

2.4 Limitations of Existing Methods

While the drift detection literature offers diverse approaches, each category has characteristic limitations:

Supervised Methods (DDM, EDDM, HDDM): According to Lu et al. (2018), these require timely labeled data, which is often delayed or unavailable in production. Sethi and Kantardzic (2017) emphasizes that they detect drift reactively—only after performance degradation has already occurred.

Statistical Distribution Methods (ADWIN, KSWIN, KL-Divergence): According to Rabanser et al. (2019), these can be sensitive to noise, leading to false alarms. Lu et al. (2018) note they may struggle in high-dimensional or correlated feature spaces. Webb et al. (2016) observe that they typically test marginal distributions, potentially missing joint distribution changes.

Prediction/Uncertainty-Based Methods: According to Guo et al. (2017), these rely on model calibration. Ovadia et al. (2019) emphasize that modern ML models are often poorly calibrated, especially under distribution shift, causing these signals to be unreliable.

Context-Aware Methods: According to Krawczyk et al. (2017), these require pre-specification of relevant context variables, which may be incomplete, noisy, or themselves drift over time. Misspecified context can cause missed detections or spurious alarms.

Fundamental Gap: According to Lopez-Paz and Oquab (2017), critically, most existing methods do not directly test whether new data is statistically distinguishable from historical data in the original feature space. Pan et al. (2020) note that discriminative approaches offer a natural, model-agnostic signal of drift that does not depend on predefined contexts, labels, or calibrated uncertainties—yet this perspective has received limited systematic attention in the drift detection literature.

This thesis aims to bridge this gap by establishing adversarial validation as a viable, general-purpose drift detection technique through rigorous empirical evaluation and practical methodology development.

2.5 Related Work: Adversarial Validation

2.5.1 Concept and Origins

According to Pan et al. (2020), adversarial validation was originally proposed as a practical technique for diagnosing dataset shift between training and evaluation data. Pan et al. (2020) and Qian and Rao (2021) explain that the approach reframes the comparison of two datasets as a binary classification problem: train a classifier to predict whether each instance originates from the reference distribution (e.g., training data) or the target distribution (e.g., test or new data). Lopez-Paz and Oquab (2017) note that high discriminative performance indicates statistical separability—hence significant distributional shift—while near-random performance suggests the datasets are largely indistinguishable.

According to Pan et al. (2020), the term “adversarial” reflects the insight that if an adversary (the classifier) cannot distinguish the data sources, the distributions must be similar. Lopez-Paz and Oquab (2017) note this is related to the discriminator component in Generative Adversarial Networks (GANs), though applied here for diagnostic rather than generative purposes.

2.5.2 Industrial Applications

Uber’s MaLTA System: Pan et al. (2020) presented adversarial validation at Uber for their MaLTA (Machine Learning Targeting Automation) user targeting system. They trained a classifier to distinguish historical training data from newly observed targeting data. When the adversarial classifier exhibited strong separability, it signaled train-deployment mismatch, prompting feature filtering and retraining adjustments before deployment. Their results demonstrated improved robustness compared to naive retraining strategies that react only after observable performance degradation.

Credit Scoring: Qian and Rao (2021) applied adversarial validation to manage dataset shift in credit scoring. They trained an adversarial classifier to distinguish historical credit applications from recent ones, using the classifier’s predictions to filter training samples—keeping only those most similar to the current deployment distribution for model training.

Image Classification and Operations: Additional applications include using adversarial validation to quantify dataset complexity in image classification Renza and Moya-Albor (2024) and to detect when historical data is no longer representative in manufacturing/operations settings Senoner et al. (2023).

2.6 Gap in the Literature

Although adversarial validation has been successfully applied to measure dataset shift in specific industrial settings, according to Pan et al. (2020), it has not been developed into a general, systematic feature drift detection framework. Specifically:

1. **No benchmark evaluation:** According to Pan et al. (2020), adversarial validation has not been systematically evaluated as a drift detector on standard drift benchmark datasets under streaming or window-based protocols.
2. **No formal framework:** Qian and Rao (2021) note that existing uses are embedded within domain-specific pipelines rather than framed as general-purpose drift monitoring techniques with clear decision rules and theoretical justification.
3. **No comparative analysis:** There is no systematic comparison of adversarial validation against established drift detection methods such as DDM (Gama et al., 2004), EDDM (Baena-García et al., 2006), ADWIN (Bifet and Gavaldà, 2007), and KSWIN (Raab et al., 2020) across multiple drift scenarios.
4. **No practical guidance:** According to Pan et al. (2020), limited guidance exists on classifier selection, window sizing, threshold calibration, and deployment considerations for adversarial validation as a drift monitor.

According to Lopez-Paz and Oquab (2017), this gap is particularly significant given that adversarial validation is fundamentally an unsupervised, discriminative drift detection method that directly tests distributional separability—addressing key limitations

of both statistical tests (which may miss high-dimensional shifts) and prediction-based methods (which rely on calibration assumptions).

This thesis aims to bridge this gap by establishing adversarial validation as a viable, general-purpose drift detection technique through rigorous empirical evaluation and practical methodology development.

CHAPTER III

METHODOLOGY

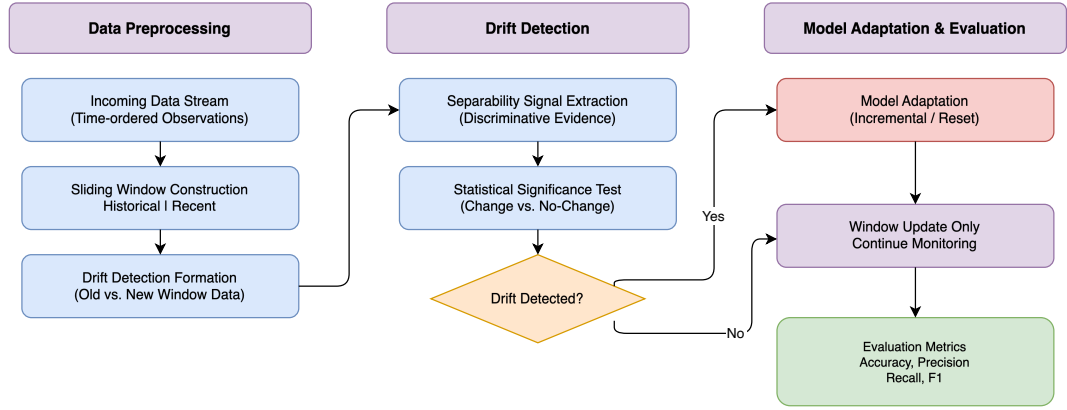


Figure 3.1: Overview of the methodology for feature drift detection via adversarial validation.

3.1 Problem Formulation

According to Pan et al. (2020) and Lopez-Paz and Oquab (2017), this research formalizes feature (covariate) drift detection as a problem of discriminating between historical reference data and newly arriving observations in a streaming or windowed setting. Shimodaira (2000) explains that the objective is to detect when the underlying feature distribution $P(X)$ changes substantially over time, providing an early indication of distributional drift before observable degradation in task performance occurs.

Formal Definition: Let $\{x_t\}_{t=1}^T$ denote a stream of feature vectors arriving over time. At each monitoring point t , a drift detector produces a binary decision $d_t \in \{\text{no-drift}, \text{drift}\}$ based on comparison between:

- A reference window W_{ref} containing historical samples presumed to represent the stable/baseline distribution

- A current window W_{cur} containing the most recent observations under test

According to Lu et al. (2018) and Gama et al. (2014), the detection task must balance:

- **Promptness:** Low detection delay (lag between true drift and detection)
- **Reliability:** Low false positive rate (false alarms when no drift exists)
- **Sensitivity:** High true positive rate (detecting drift when it occurs)
- **Efficiency:** Bounded computational and memory requirements suitable for production

3.2 Core Components of the Framework

The proposed methodology consists of five core components:

1. Windowing Strategy: Managing data streams via sliding or adaptive windows
2. Adversarial Classifier: Training a discriminative model between reference and current distributions
3. Detection Statistic and Decision Rule: Quantifying separability to generate drift signals
4. Model Adaptation Strategy: Triggering retraining or updates upon drift detection
5. Evaluation Protocol: Systematic metrics and benchmarks for assessment

3.2.1 Streaming Window Management

Consistent with established streaming drift detection methodologies (Bifet and Gavaldà, 2007; Tsymbal, 2004), a sliding window mechanism manages the data stream:

Reference Window (W_{ref}): A buffer containing N_{ref} historical samples representing the stable distribution. This window may be:

- Fixed to initial training data
- Periodically updated with a sliding approach
- Adaptively resized based on detection signals (Bifet and Gavalda, 2007)

Current Window (W_{cur}): According to Bifet and Gavalda (2007), a buffer containing the N_{cur} most recent observations under test. As new data arrives, older samples are discarded to maintain responsiveness.

Update Mechanism: At each monitoring step, windows slide forward:

- New samples enter W_{cur}
- Oldest samples exit W_{cur} (optionally entering W_{ref} if using sliding reference)
- Upon drift detection, W_{ref} may be reset to current data

According to Lu et al. (2018), window sizes represent a bias-variance tradeoff: larger windows provide more stable estimates but slower response; smaller windows enable faster detection but higher variance.

3.2.2 Adversarial Validation as Discriminative Drift Detector

According to Pan et al. (2020), the core contribution is adapting adversarial validation to drift detection:

Step 1: Construct Binary Classification Dataset

- Label all samples from W_{ref} as class 0 (reference)
- Label all samples from W_{cur} as class 1 (current)
- Combined dataset: $X = W_{ref} \cup W_{cur}$, $y = \{0\}^{N_{ref}} \cup \{1\}^{N_{cur}}$

Step 2: Train Discriminative Classifier

According to Lopez-Paz and Oquab (2017), train a binary classifier f_θ to predict data source from features alone. Candidate classifiers include:

- Logistic regression (interpretable, fast)
- Random forest (handles non-linearity, feature interactions)

- Gradient boosting (XGBoost, LightGBM—high accuracy)
- Neural network (for high-dimensional data)

Step 3: Measure Discriminative Performance

According to Pan et al. (2020), evaluate classifier performance using held-out validation or cross-validation:

- AUC-ROC: Area under the ROC curve (recommended primary metric)
- Accuracy: Classification accuracy
- AUC-PR: Area under the precision-recall curve (for imbalanced windows)

Interpretation:

- $AUC \approx 0.5$: Classifier cannot distinguish sources $\rightarrow$ distributions are similar $\rightarrow$ no drift
- $AUC \gg 0.5$: Classifier distinguishes sources well $\rightarrow$ distributions differ $\rightarrow$ drift detected
- According to Lopez-Paz and Oquab (2017), the magnitude of AUC indicates drift severity

3.2.3 Detection Statistic and Decision Rules

The separation score serves as the drift statistic. Let $S_t = \text{AUC}(f_\theta, W_{ref}, W_{cur})$ at monitoring time t .

Simple Threshold Rule: According to Pan et al. (2020), raise drift alarm at time t if: $S_t > T_{drift}$, where T_{drift} is a threshold chosen based on desired sensitivity and false alarm rate.

Permutation Test for Statistical Significance: According to Lopez-Paz and Oquab (2017), to obtain principled thresholds:

1. Shuffle source labels randomly B times (e.g., $B = 100$)
2. Retrain classifier and compute AUC for each permutation
3. Construct empirical null distribution of AUC under no-drift hypothesis

4. Set threshold at $(1 - \alpha)$ quantile for significance level α

Consecutive Confirmation: According to Gama et al. (2004), to reduce false alarms, require drift signal to persist: raise alarm only if $S_t > T_{drift}$ for k consecutive monitoring steps.

3.2.4 Model Adaptation Strategies

According to Gama et al. (2014), upon drift detection, the primary prediction model can be adapted:

Strategy 1: Drift-Triggered Retraining

- Model is trained at initialization
- Retraining occurs only when drift alarm is raised
- Reference window resets to current data after retraining
- According to Pan et al. (2020), advantage: Conserves resources; exploits early detection for timely updates

Strategy 2: Periodic Retraining (Baseline)

- Model is retrained at fixed intervals regardless of drift signals
- Advantage: Simple; does not depend on detector reliability
- According to Gama et al. (2014), disadvantage: May retrain unnecessarily or too late

Strategy 3: Continuous Update with Drift Weighting

- Online learning with sample weighting based on drift score
- According to Krawczyk et al. (2017), recent samples receive higher weights when drift is detected

3.3 Datasets

According to Souza et al. (2020), experiments will use both synthetic and real-world benchmark datasets commonly employed in drift detection research:

Synthetic Datasets:

- SEA Concepts (Street and Kim, 2001): Classic drift benchmark with abrupt concept drift
- Rotating Hyperplane: According to Lu et al. (2018), incremental drift through gradual rotation
- STAGGER Concepts: According to Gama et al. (2014), abrupt drift between different rule sets
- Sine/Circle generators: According to Souza et al. (2020), controlled covariate drift scenarios
- Custom synthetic data: Controlled $P(X)$ shifts with known ground truth

Real-World Benchmark Datasets:

- Electricity (Elec2) (Harries, 1999): Electricity price prediction with natural drift
- Airline: According to Souza et al. (2020), flight delay prediction with temporal drift
- Forest Covertype: Land cover classification with spatial/temporal variation
- Credit Card Fraud: Transaction data with evolving fraud patterns
- Gas Sensor Array Drift (Vergara et al., 2012): Sensor readings with documented drift

According to Souza et al. (2020), datasets will be processed in streaming fashion with features used for drift detection and labels reserved for downstream performance evaluation.

3.4 Baseline Drift Detectors

Adversarial validation will be compared against established baselines:

Supervised Baselines:

- DDM (Drift Detection Method) Gama et al. (2004)
- EDDM (Early Drift Detection Method) Baena-García et al. (2006)
- HDDM-A/W (Hoeffding Drift Detection Method) Frias-Blanco et al. (2015)
- Page-Hinkley Test Page (1954)

Unsupervised Statistical Baselines:

- ADWIN (Adaptive Windowing) Bifet and Gavalda (2007)
- KSWIN (Kolmogorov-Smirnov Windowing) Raab et al. (2020)
- MMD (Maximum Mean Discrepancy) two-sample test Gretton et al. (2012)
- Chi-squared test on discretized features

Unsupervised Model-Based Baselines:

- D3 (Discriminative Drift Detector) Sethi and Kantardzic (2017) if implementation available
- Margin density-based detectors Ditzler and Polikar (2011)

3.5 Evaluation Metrics

According to Lu et al. (2018) and Souza et al. (2020), **Drift Detection Performance** metrics include:

- Detection Delay: Time between true drift occurrence and detection
- False Positive Rate (FPR): Proportion of alarms when no drift exists
- True Positive Rate (TPR) / Sensitivity: Proportion of true drifts detected
- Precision: Proportion of alarms that are true drifts
- F1-Score: Harmonic mean of precision and recall
- AUC of Detection: Summary measure of detection performance

Model Performance Impact:

- Prequential Accuracy/F1: According to Gama et al. (2014), online accuracy using test-then-train protocol
- Recovery Time: Time to restore model performance after drift
- Mean Squared Error / RMSE: For regression tasks

3.6 Experimental Design**Experiment 1: Sensitivity to Drift Types**

According to Lu et al. (2018), evaluate detection performance across abrupt, gradual, incremental, and recurring drift patterns on synthetic data with known drift points.

Experiment 2: Real-World Benchmark Comparison

According to Souza et al. (2020), compare detection timeliness and model performance across real-world datasets against all baseline methods.

Experiment 3: Hyperparameter Sensitivity

Analyze impact of:

- Window sizes (N_{ref} , N_{cur})
- Classifier choice (logistic regression, random forest, XGBoost)
- Detection threshold (T_{drift})
- Monitoring frequency

Experiment 4: Scalability and Efficiency

Measure computational time and memory usage across varying data dimensionality and stream length.

Experiment 5: Adaptation Strategy Comparison

According to Gama et al. (2014), compare drift-triggered retraining vs. periodic retraining vs. no retraining in terms of downstream model performance.

3.7 Practical Considerations

Class Imbalance: When $|W_{ref}| \neq |W_{cur}|$, use:

- Stratified cross-validation
- Class-weighted classifiers
- According to Lopez-Paz and Oquab (2017), AUC-ROC rather than accuracy

Computational Efficiency:

- Use efficient classifiers (logistic regression, small random forests)
- Consider approximate methods for large windows
- According to Pan et al. (2020), implement incremental training where possible

Temporal Leakage Prevention:

- Use purged cross-validation to avoid temporal overlap between folds
- According to Souza et al. (2020), ensure strict temporal ordering in train/test splits

Threshold Calibration:

- Use validation data with known drift to tune T_{drift}
- According to Pan et al. (2020), consider adaptive thresholds based on baseline variance

CHAPTER IV

RESULTS

CHAPTER V

DISCUSSION

CHAPTER VI

CONCLUSION

To ensure empirical reproducibility and practical adoption, all experiments, data preprocessing scripts, synthetic data generators, and evaluation harnesses will be published in a public repository (GitHub) alongside a versioned environment specification (e.g., `requirements.txt` or `pyproject.toml`). Experiments will include fixed random seeds, explicit train/validation/test splits, and a short README that documents how to reproduce each experiment and the compute resources used. Where public benchmark datasets require licensing or restricted access, the repository will include instructions and pointers rather than redistributing raw data.

Ethical considerations are important for drift detection research: (1) monitor and report any privacy-sensitive features used in experiments and remove or mask personally identifiable information (PII) from shared artifacts; (2) analyze potential harms from automated retraining decisions (e.g., model bias amplification) and recommend safe human-in-the-loop review for high-risk applications (credit, healthcare); (3) document limitations of detectors (false positives/negatives) and provide operational guidance for threshold selection and escalation policies.

Conclusion and Timeline

This thesis develops adversarial validation into a systematic, practical framework for feature (covariate) drift detection in streaming and windowed settings. Contributions include (a) a reproducible adversarial-validation detector that combines ROC-AUC effect-size with permutation-test p-values for robust decisions; (b) a comprehensive benchmark against supervised and unsupervised baselines across synthetic and real-world datasets; and (c) practical deployment guidance for MLOps pipelines.

Proposed timeline (high-level):

- Months 1–2: finalize dataset selection, implement streaming harness and baseline detectors.
- Months 3–5: implement adversarial-validation detector, tune hyperparameters, and run synthetic experiments.
- Months 6–8: run real-world benchmarks, analyze detection delay and model recovery, perform ablations.
- Months 9–10: write up results, prepare reproducible code release, and draft thesis chapters.

Deliverables: reproducible code repository, benchmark results, and the thesis document with practical recommendations for deploying adversarial validation in production.

REFERENCES

- Baena-García, M., del Campo-Ávila, J., Fidalgo, R., Bifet, A., Gavaldà, R., and Morales-Bueno, R. (2006). Early drift detection method. In *Proceedings of the Fourth International Workshop on Knowledge Discovery from Data Streams (IWKDDs)*, pages 77–86.
- Bayram, F., Ahmed, B. S., and Kassler, A. (2022). From concept drift to model degradation: An overview on performance-aware drift detectors. *Knowledge-Based Systems*, 245:108632.
- Bifet, A. and Gavaldà, R. (2007). Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining (SDM)*, pages 443–448. SIAM.
- Dasu, T., Krishnan, S., Venkatasubramanian, S., and Yi, K. (2006). An information-theoretic approach to detecting changes in multi-dimensional data streams. In *Proceedings of the 38th Symposium on the Interface of Statistics, Computing Science, and Applications*.
- Ditzler, G. and Polikar, R. (2011). Hellinger distance based drift detection for nonstationary environments. In *IEEE Symposium on Computational Intelligence in Dynamic and Uncertain Environments (CIDUE)*, pages 41–48.
- Frias-Blanco, I., del Campo-Ávila, J., Ramos-Jiménez, G., Morales-Bueno, R., Ortiz-Díaz, A., and Caballero-Mota, Y. (2015). Online and non-parametric drift detection methods based on Hoeffding’s bounds. *IEEE Transactions on Knowledge and Data Engineering*, 27(3):810–823.
- Gama, J., Medas, P., Castillo, G., and Rodrigues, P. (2004). Learning with drift detection. In *Advances in Artificial Intelligence – SBIA 2004*, volume 3171 of *LNCS*, pages 286–295. Springer.

- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37.
- Gözüaık, Ö., Büyükakır, A., Bonab, H., and Can, F. (2019). Unsupervised concept drift detection with a discriminative classifier. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 2365–2368.
- Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B., and Smola, A. (2012). A kernel two-sample test. *Journal of Machine Learning Research*, 13(25):723–773.
- Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *PMLR*, pages 1321–1330.
- Harries, M. (1999). Splice-2 comparative evaluation: Electricity pricing. Technical report, University of New South Wales.
- Hinder, F., Vaquet, V., Brinkrolf, J., and Hammer, B. (2024). One or two things we know about concept drift—a survey on monitoring in evolving environments. *Frontiers in Artificial Intelligence*, 7:1330257.
- Krawczyk, B., Minku, L. L., Gama, J., Stefanowski, J., and Woźniak, M. (2017). Ensemble learning for data stream analysis: A survey. *Information Fusion*, 37:132–156.
- Liu, F., Xu, W., Lu, J., Zhang, G., Gretton, A., and Sutherland, D. J. (2020). Learning deep kernels for non-parametric two-sample tests. In *International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 6316–6326.
- Long, M., Zhu, H., Wang, J., and Jordan, M. I. (2016). Unsupervised domain adaptation with residual transfer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29.
- Lopez-Paz, D. and Oquab, M. (2017). Revisiting classifier two-sample tests. In *International Conference on Learning Representations (ICLR)*.

- Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., and Zhang, G. (2018). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12):2346–2363.
- Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J., Lakshminarayanan, B., and Snoek, J. (2019). Can you trust your model’s uncertainty? evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 13991–14002.
- Page, E. S. (1954). Continuous inspection schemes. *Biometrika*, 41(1/2):100–115.
- Pan, J., Pham, V., Doroudian, M., and Arora, N. (2020). MaLTA: Machine learning for user targeting at uber. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 3475–3483.
- Qian, H. and Rao, S. (2021). Managing dataset shift by adversarial validation for credit scoring. *arXiv preprint arXiv:2112.04471*.
- Raab, C., Heusinger, M., and Schleif, F.-M. (2020). Reactive soft prototype computing for concept drift streams. *Neurocomputing*, 416:340–351.
- Rabanser, S., Günnemann, S., and Lipton, Z. C. (2019). Failing loudly: An empirical study of methods for detecting dataset shift. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 1396–1408.
- Renza, D. and Moya-Albor, E. (2024). Adversarial validation approach to concept drift detection in automated machine learning. *Applied Sciences*, 14(3):1178.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NIPS)*, pages 2503–2511.
- Senoner, J., Netland, T. H., and Korherr, B. (2023). Using machine learning to predict and monitor operational performance in order picking. *International Journal of Production Research*, 61(23):8207–8227.

- Sethi, T. S. and Kantardzic, M. (2017). On the reliable detection of concept drift from streaming unlabeled data. *Expert Systems with Applications*, 82:77–99.
- Shimodaira, H. (2000). Improving predictive inference under covariate shift by weighting the log-likelihood function. *Journal of Statistical Planning and Inference*, 90(2):227–244.
- Souza, V. M. A., Reis, D. M., Maletzke, A. G., and Batista, G. E. A. P. A. (2020). Challenges in benchmarking stream learning algorithms with real-world data. *Data Mining and Knowledge Discovery*, 34(6):1805–1858.
- Street, W. N. and Kim, Y. (2001). A streaming ensemble algorithm (SEA) for large-scale classification. In *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 377–382.
- Tsymbol, A. (2004). The problem of concept drift: Definitions and related work. Technical Report TCD-CS-2004-15, Trinity College Dublin, Department of Computer Science.
- Vergara, A., Vembu, S., Ayhan, T., Ryan, M. A., Homer, M. L., and Huerta, R. (2012). Chemical gas sensor drift compensation using classifier ensembles. *Sensors and Actuators B: Chemical*, 166:320–329.
- Webb, G. I., Hyde, R., Cao, H., Nguyen, H. L., and Petitjean, F. (2016). Characterizing concept drift. *Data Mining and Knowledge Discovery*, 30(4):964–994.
- Widmer, G. and Kubat, M. (1996). Learning in the presence of concept drift and hidden contexts. *Machine Learning*, 23(1):69–101.
- Žliobaitė, I. (2010). Learning under concept drift: An overview. *arXiv preprint arXiv:1010.4784*.

BIOGRAPHY

NAME	Ms. Nusrat Begum
DATE OF BIRTH	21 February 1998
PLACE OF BIRTH	Chattogram, Bangladesh
INSTITUTIONS ATTENDED	Stamford International University, 2017–2022 Bachelor of International Business Management () Mahidol University, 2024–2026 Master of Science (Computer Science) Mahidol University
E-MAIL	nusrat.beg@student.mahidol.edu